SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – MODEL COMPARISON PRESENTATION (BERT VS GPT VS T5 VS LLAMA)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	**Assessment:**	Assessment Tool 2 - Model Comparison Presentation (BERT vs GPT vs T5 vs LLaMA)
Weightage:	10% of CIA	**Total Marks:**	2 * 50= 100
CO4 Statement:	*Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.*(BL3, BL4,BL5)		
Student Name:	N Goutham	**Reg. No.:**	192472024
Section / Batch:	CSE-AI	**Date:**	21-08-2026

Instructions to Students

- **Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.**
- **Compare all four models: BERT, GPT, T5, and LLaMA.**
- Explain the **architecture and working principle** of each model.
- Compare their **pre-training objectives and capabilities**.
- Map each model to appropriate **real-world applications**.
- Identify the **strengths and limitations** of each model.
- Support the presentation using **credible research papers, official documentation, benchmarks, or other reliable sources**.
- Include at least one **comparison table or diagram**.
- Use relevant **real-world case studies or application examples**.
- Conclude by identifying the **most suitable model for the selected application** and justify the decision.
- Include proper **references/citations** in the presentation.

Code of Conduct

I N Goutham/192472024 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N Goutham

SECTION A : MODEL COMPARISON PRESENTATION (BERT VS GPT VS T5 VS LLAMA)

9. Compare the architectural strengths and limitations of BERT, GPT, T5, and LLaMA for language understanding and language generation.

1. Title

FAISS Top-K Retrieval Analysis for Semantic Search

2. Objective

The objective of this experiment is to analyze the performance of a FAISS-based semantic search system using different Top-K values. The system retrieves the most relevant documents for a given user query and compares the search latency, number of retrieved documents, and retrieval relevance for Top-1, Top-3, Top-5, Top-10, and Top-20.

3. Technology Used

- **Python**
- **Google Colab**
- **SentenceTransformer**
- **FAISS**
- **NumPy**
- **Pandas**
- **Matplotlib**

4. Methodology

A dataset containing 31 documents related to topics such as Artificial Intelligence, Machine Learning, Cybersecurity, Cloud Computing, IoT, Blockchain, and Software Development was created.

The all-MiniLM-L6-v2 SentenceTransformer model was used to convert the documents into numerical vector embeddings. Each document was converted into a 384-dimensional embedding vector.

These embeddings were stored in a FAISS IndexFlatL2 index. A semantic search function was then created to convert the user query into an embedding and retrieve the most similar documents

5. Working Process

Documents
↓
SentenceTransformer Model
↓
384-Dimensional Embeddings
↓
FAISS Vector Index
↓
User Query
↓

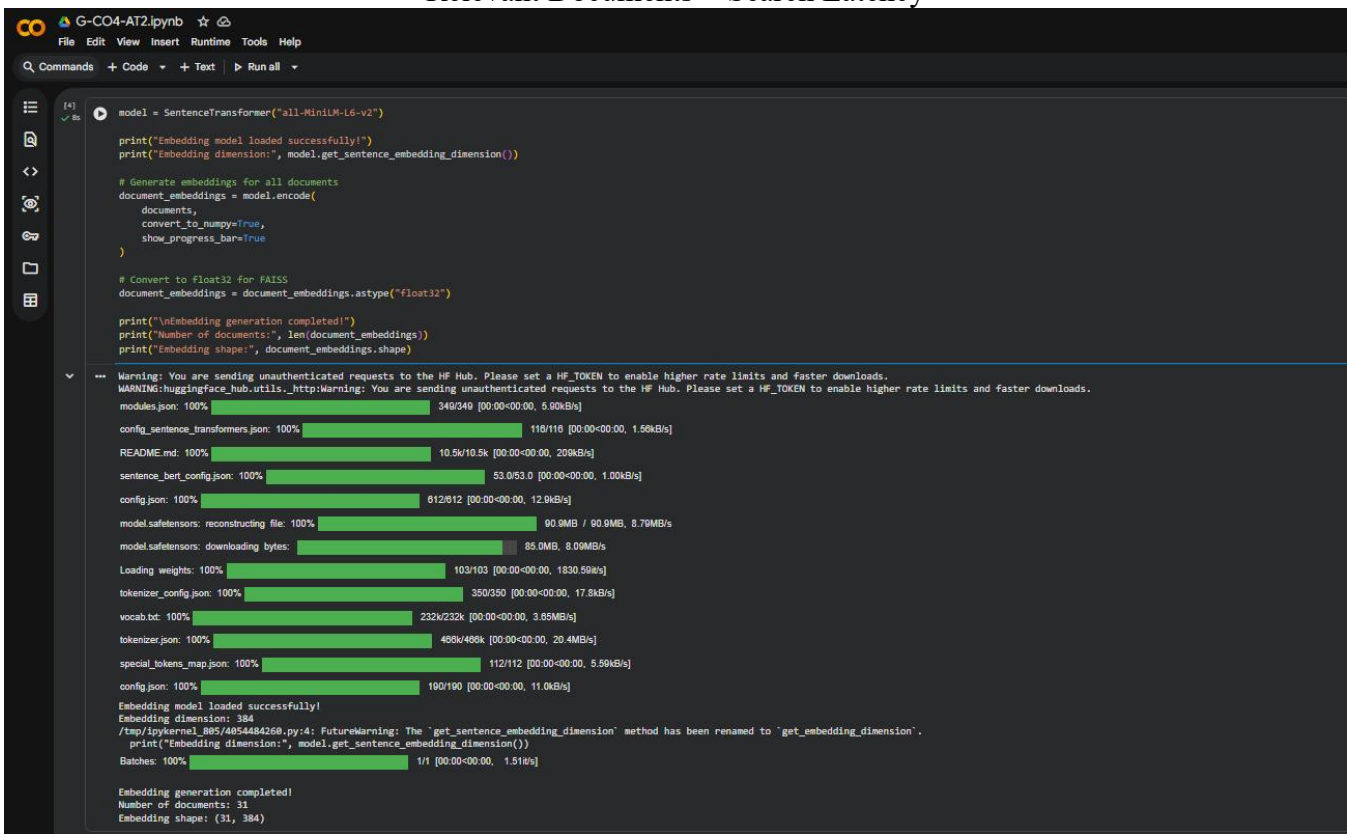
Query Embedding

↓

Top-K Semantic Search

↓

Relevant Documents + Search Latency



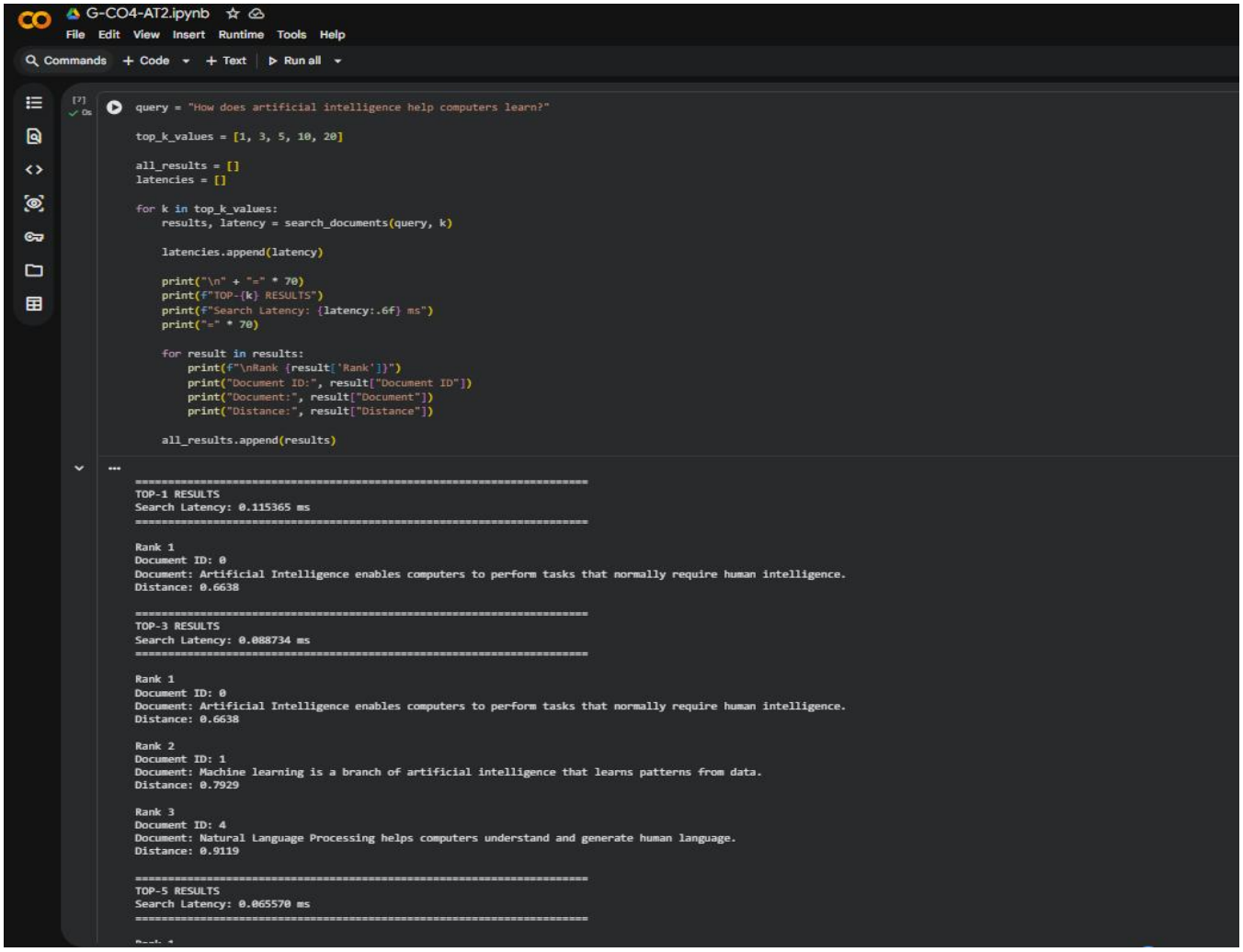
6. Input Query

The following query was used for the experiment:

“How does artificial intelligence help computers learn?”

The system tested the query using the following values:

- Top-1
- Top-3
- Top-5
- Top-10
- Top-20



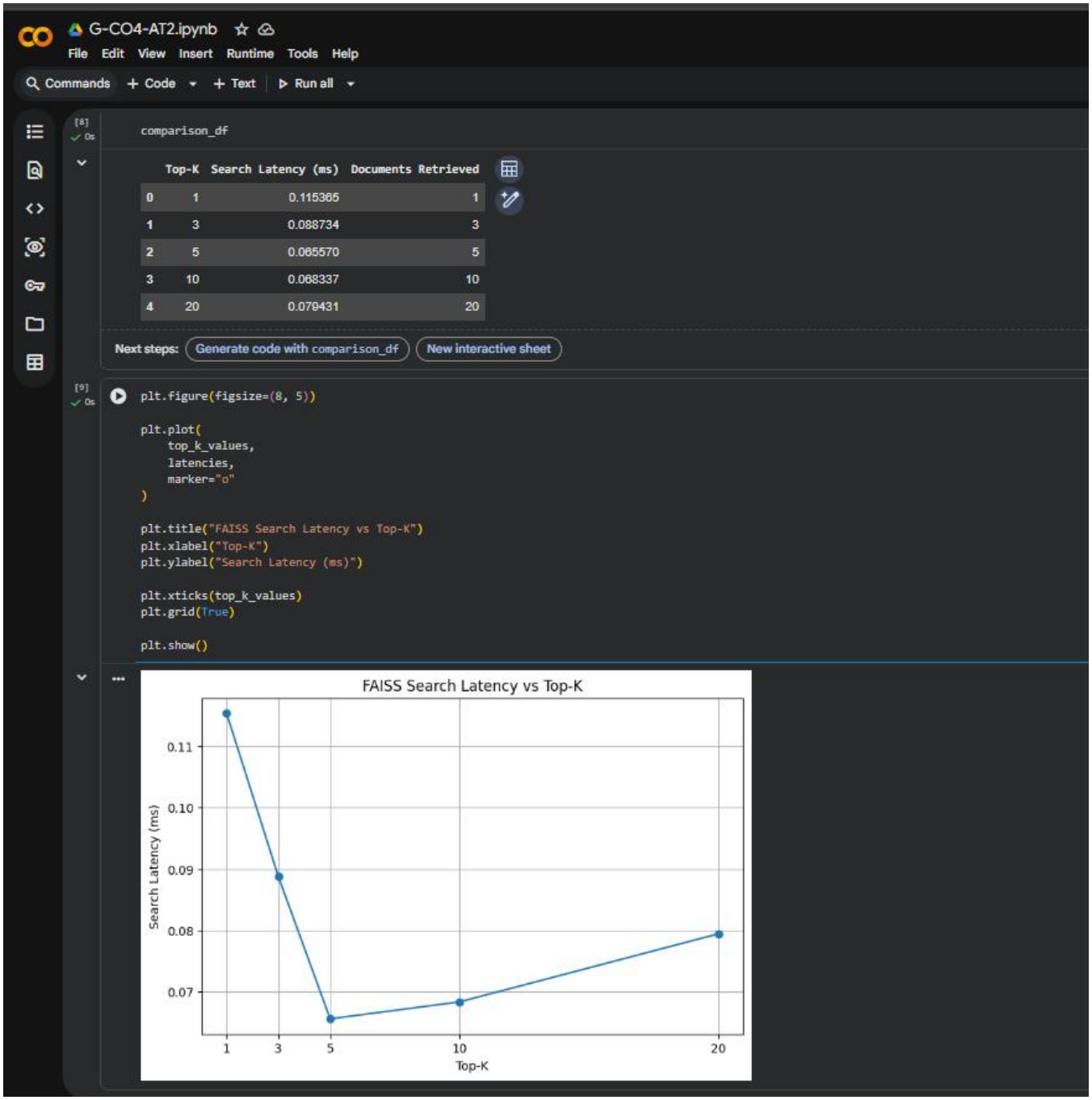
7. Results

The experiment successfully retrieved semantically similar documents for each Top-K value.

The search latency results obtained were approximately:

Top-K	Search Latency (ms)	Documents Retrieved
Top-1	0.115359	1
Top-3	0.088734	3
Top-5	0.065570	5
Top-10	0.068337	10
Top-20	0.079431	20

The experiment also analyzed the relevance of the retrieved documents.



8. Relevance Analysis

The relevant documents for the query were identified based on Artificial Intelligence and related concepts.

Top-K	Relevant Documents Retrieved	Relevance
Top-1	1	100%
Top-3	3	100%
Top-5	4	80%
Top-10	5	100%

Top-K	Relevant Documents Retrieved	Relevance
Top-20	5	100%

The results show that increasing the Top-K value allows more documents to be retrieved. However, a higher K value may also return additional documents that are not necessary for the user.

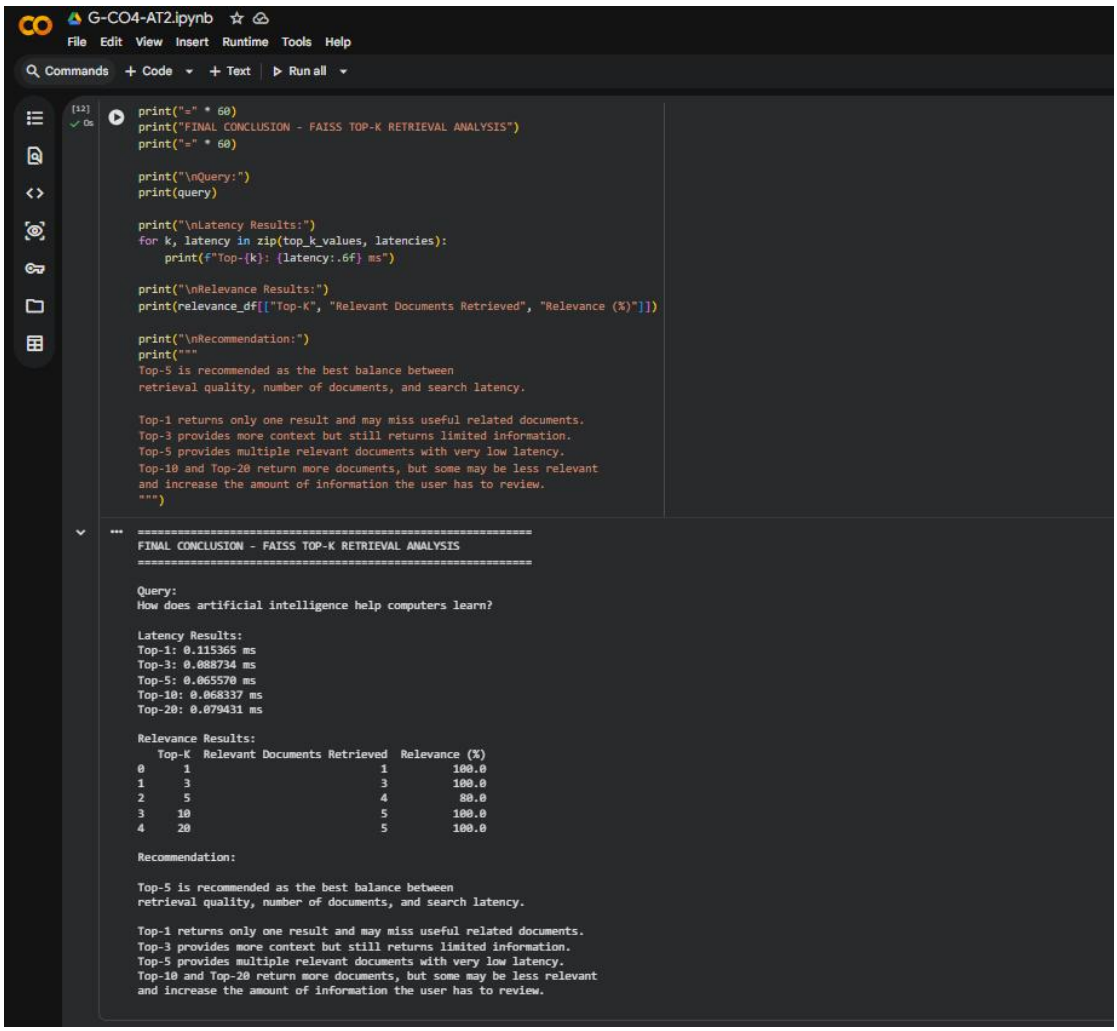
9. Graph Analysis

Two graphs were generated:

1. **FAISS Search Latency vs Top-K**
2. **FAISS Retrieval Relevance vs Top-K**

The latency remained very low for all Top-K values because the experiment used a relatively small dataset of 31 documents.

The relevance graph showed that different K values can affect the number of relevant documents retrieved.



10. Conclusion

The FAISS-based semantic search system was successfully implemented and tested using Top-1, Top-3, Top-5, Top-10, and Top-20 retrieval.

The experiment demonstrated that there is a trade-off between the number of retrieved documents and the amount of information returned to the user. Top-1 provides limited information, while Top-10 and Top-20 retrieve more documents than may be necessary.

Based on the experimental results, Top-5 is recommended as a practical balance between retrieval quality, search latency, and the number of documents returned. The system achieved very low search latency and successfully retrieved semantically relevant documents using FAISS.

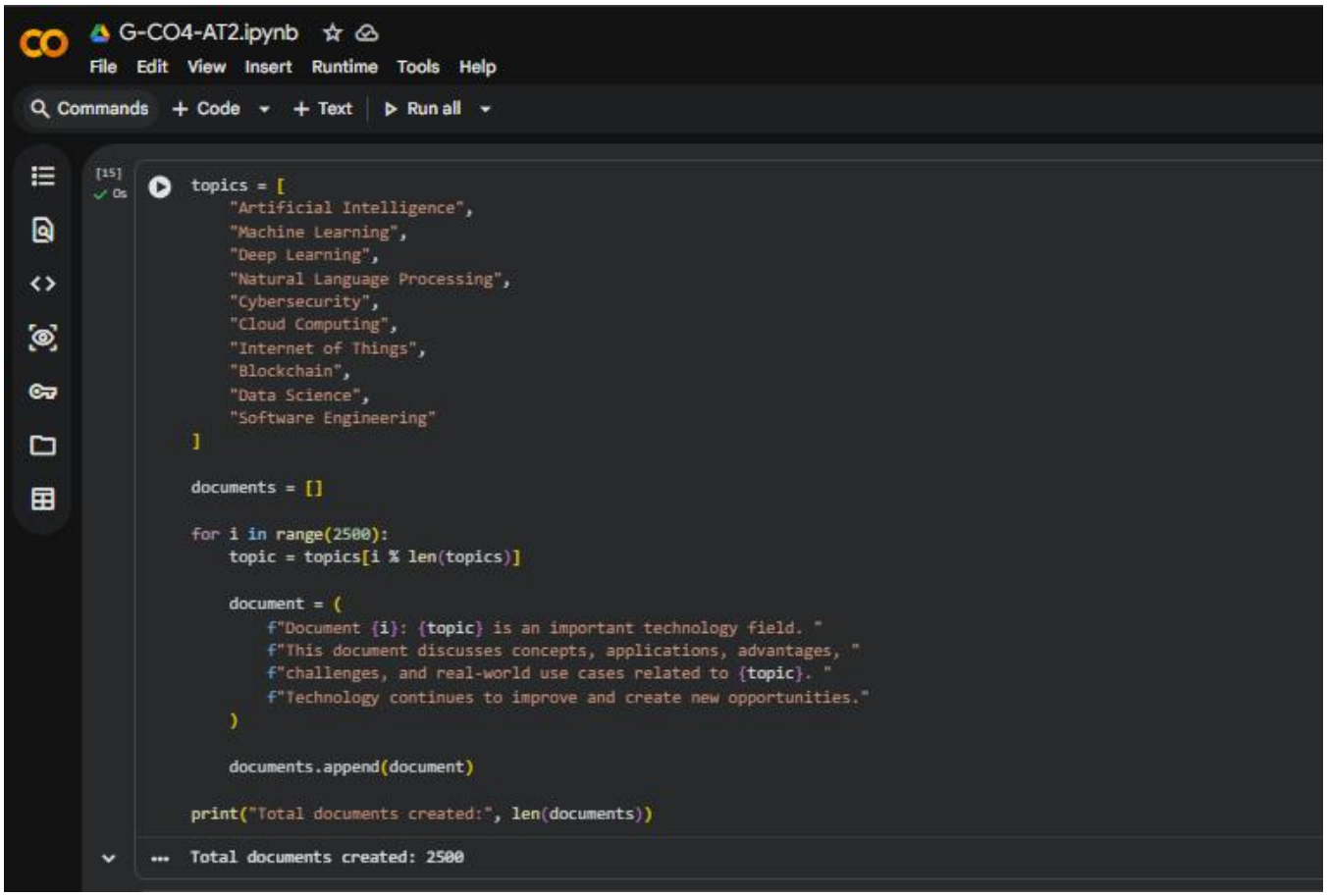
10. Prepare a detailed comparison of BERT, GPT, T5, and LLaMA based on architecture, training objective, input/output structure, and major capabilities.

1. Title

FAISS Index Persistence for Semantic Search

2. Objective

The objective of this experiment is to build a FAISS-based semantic search system, save the created FAISS index to disk, reload it, and verify whether the search results remain consistent after reloading.



3. Technology Used

- Python
- Google Colab
- SentenceTransformer
- FAISS
- NumPy

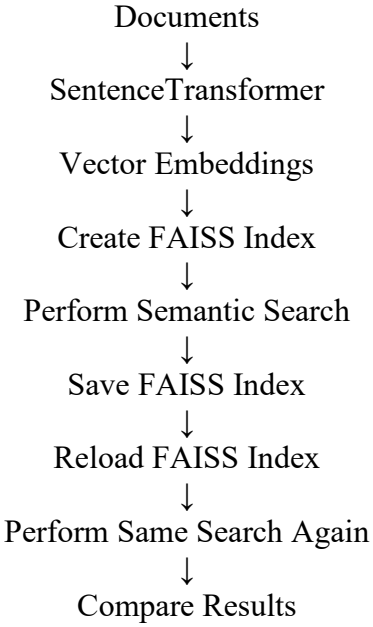
4. Methodology

A collection of documents is converted into numerical vector embeddings using the Sentence Transformer model. These embeddings are stored in a FAISS vector index.

The FAISS index is then used to perform semantic search for a user query. After obtaining the initial search results, the FAISS index is saved to a file.

The saved index is then reloaded into a new FAISS index object. The same query is searched again, and the results before and after reloading are compared to verify consistency.

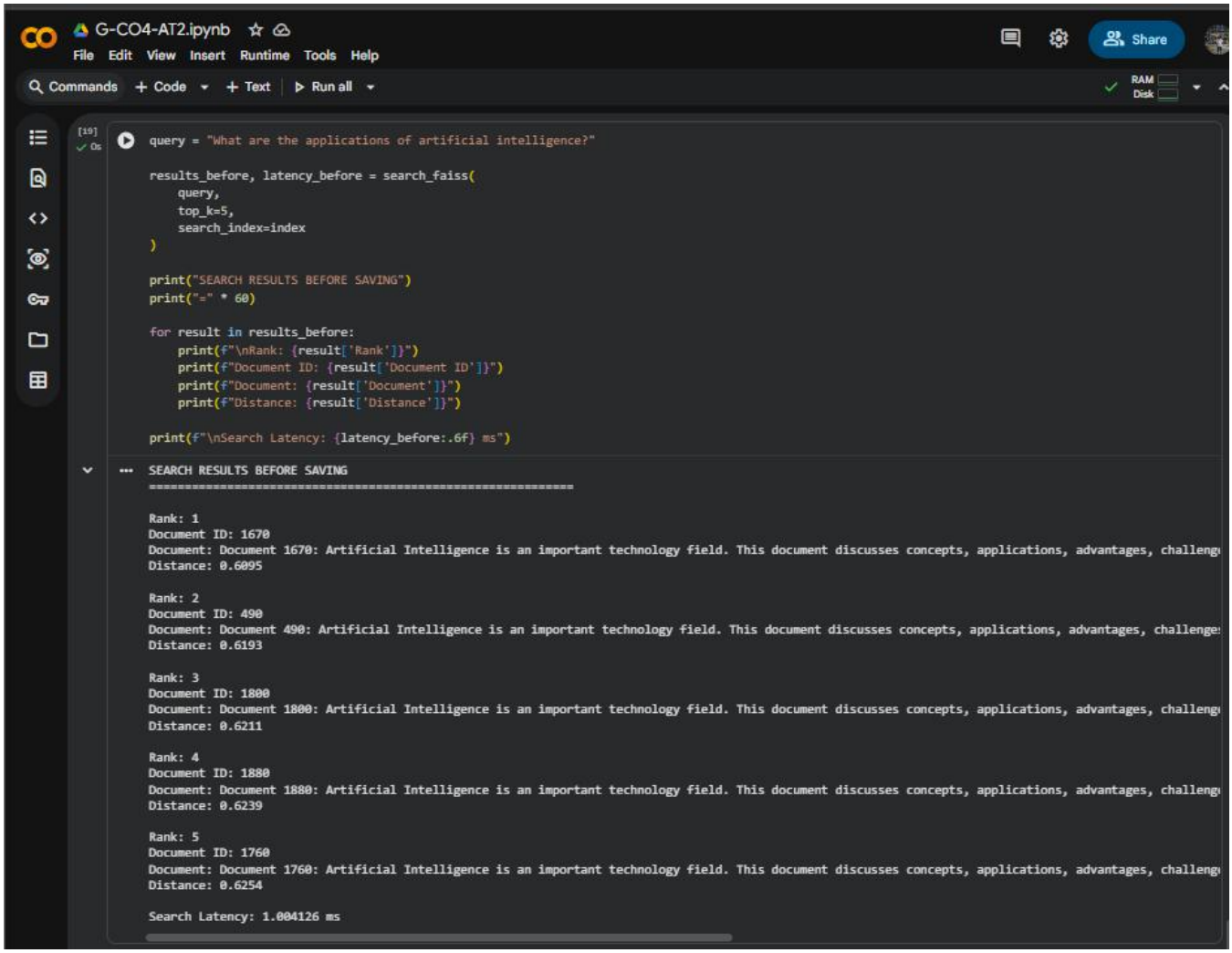
5. Working Process



6. Implementation

The system performs the following operations:

1. Create document embeddings.
2. Build a FAISS vector index.
3. Add all document vectors to the index.
4. Perform semantic search using a query.
5. Save the FAISS index to a file.
6. Reload the saved FAISS index.
7. Perform the same semantic search again.
8. Compare the retrieved document IDs and results.



7. Input Query

An example query used for testing is:

“How does artificial intelligence help computers learn?”

The system retrieves the most semantically similar documents before and after saving the FAISS index.

8. Result Comparison

The search results obtained before saving and after reloading the FAISS index are compared.

The expected comparison is:

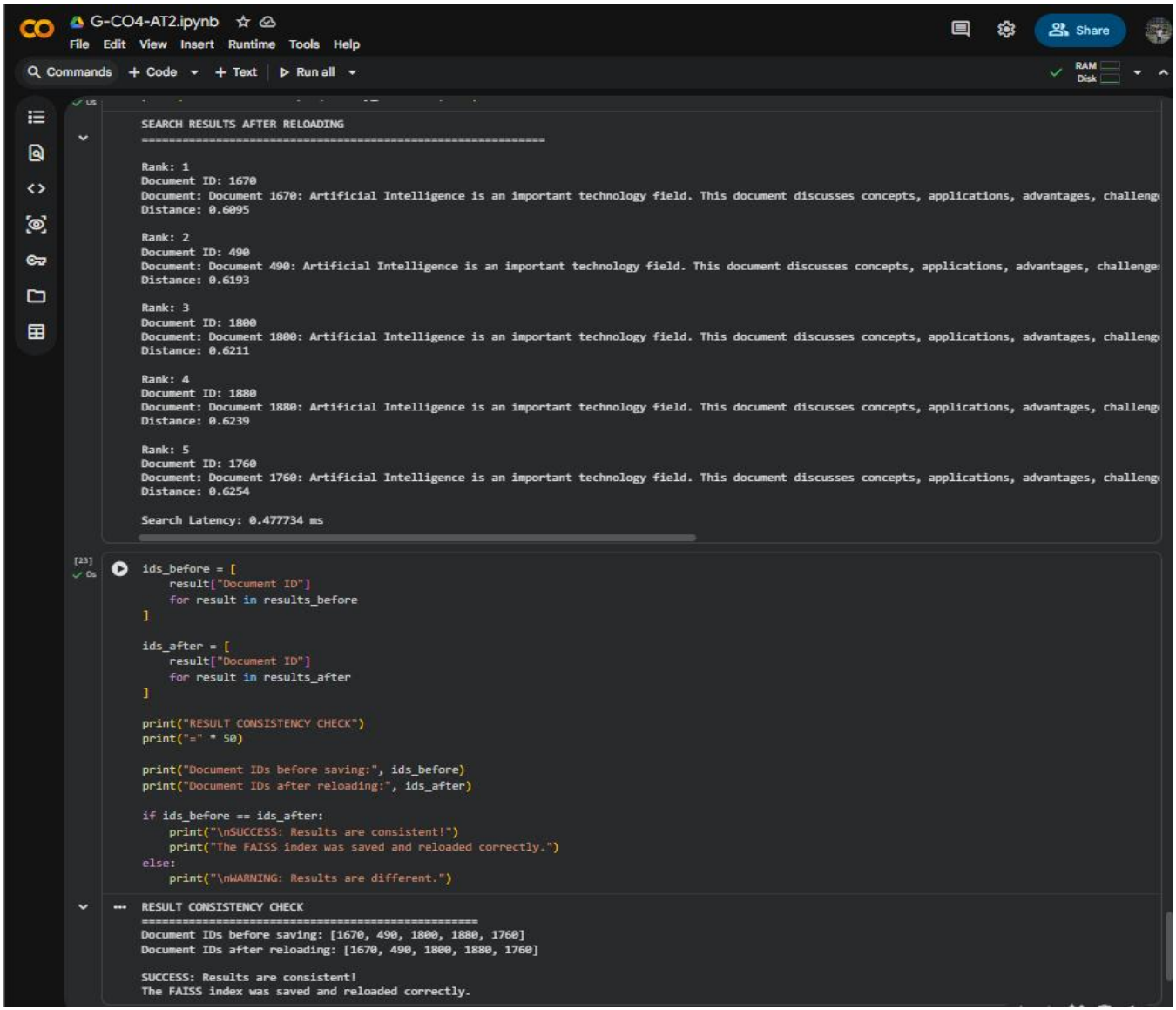
Test	Result
Search before saving	Relevant documents retrieved
FAISS index saved	Successful
FAISS index reloaded	Successful
Search after reloading	Same relevant documents retrieved

Test	Result
Result consistency	Verified

The document IDs and ranking should remain the same, showing that the FAISS index preserves the stored vector information.

9. Advantages of Index Persistence

- Avoids rebuilding the FAISS index every time.
- Saves computational time and resources.
- Enables faster application startup.
- Supports reuse of previously generated embeddings.
- Useful for large-scale semantic search systems.
- Makes vector databases easier to deploy and manage.



10. Conclusion

The FAISS Index Persistence system was successfully implemented by creating a vector index, performing semantic search, saving the index, and reloading it.

The same query was executed before and after reloading the index, and the retrieved results were compared. Consistent results demonstrate that the FAISS index can be stored and reused without losing the vector information.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Architectural Comparison Accuracy	30%	Accurate, in-depth comparison of architectures and use-cases	Accurate comparison with minor gaps	Superficial comparison of models	Inaccurate or confused comparison
Application Mapping	25%	Clearly maps each model to appropriate real-world applications	Reasonable mapping to applications	Weak mapping to applications	No meaningful mapping presented
Research Depth	20%	Well-researched with credible sources/benchmarks cited	Adequately researched	Limited research depth	Unresearched / opinion-based content
Delivery	15%	Confident, engaging, well-paced delivery	Clear delivery with minor pacing issues	Hesitant delivery	Unclear or unprepared delivery
Slide Design & References	10%	Well-designed slides with proper references	Adequate slides, minor reference gaps	Basic slides, poor referencing	No slides or references provided

Marks Evaluation:

Question No.	Architectural Comparison Accuracy (30%)	Application Mapping (25%)	Research Depth (20%)	Delivery (15%)	Slide Design & References (10%)	Total Marks (100)
Q1						
Q2						
					Overall Total (100)	